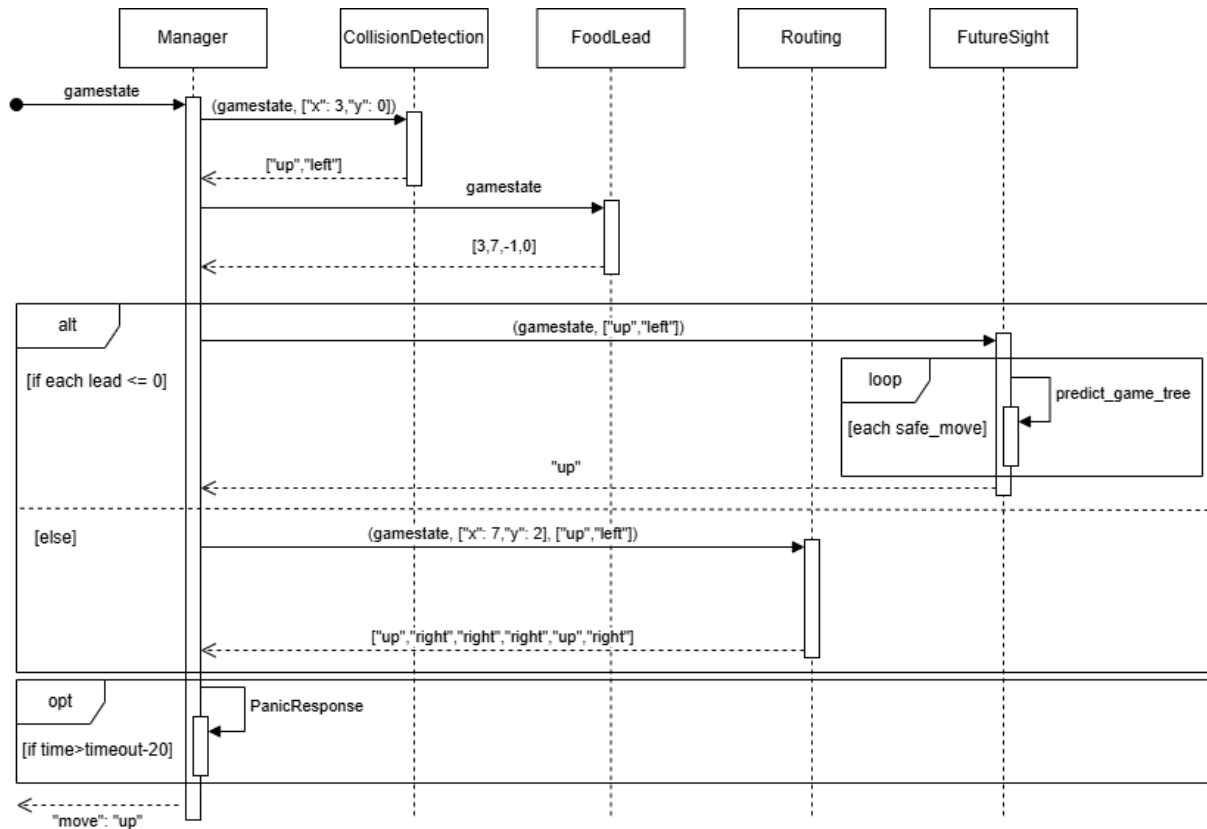


Ich musste zum Teil raten, welche Konzepte/Algorithmen genutzt werden, um die einzelnen Funktionen umzusetzen, kann also sein, dass die Beschreibungen hier nicht ganz zu der wirklichen Umsetzung passen. Sollte irgendwem ein Fehler auffallen, kann dieser Fehler gerne behoben werden. Herausforderungen fehlen aktuell noch komplett, also wenn es Schwierigkeiten bei der Umsetzung von Strategien gab, gerne in den entsprechenden Abschnitt des Dokuments schreiben.



Die zentralen Verarbeitungsschritte des Bots können (größtenteils) aus dem Sequenzdiagramm entnommen werden:

Zunächst wird der empfangene GameState etwas eingekürzt, indem irrelevante Informationen entfernt werden.

Im Folgenden ruft der Manager die Funktionen CollisionDetection und FoodLead auf und wählt auf Basis der Ausgaben die beste Strategie. Auf diese Weise wird das Teilproblem „Strategiewechsel“ gelöst.

CollisionDetection löst das Teilproblem „einfache Kollisionsprüfung“, indem die Felder aus dem GameState, die dem eigenen Schlangenkopf am nächsten sind auf Rand und andere Schlangenkörper und -köpfe überprüft und entsprechende Aktionen als (sehr) sicher oder unsicher definiert werden.

Die Funktionen FoodLead und Routing lösen das Teilproblem „gezielte (und sichere) Nahrungssuche“. FoodLead bestimmt hierbei aus Futter- und Schlangenkopfpositionen im GameState, wie viele Felder Vorsprung die eigene Schlange vor der schnellsten gegnerischen Schlange hat. Routing verwendet die Distanz aller Felder zum Ziel und gegebenenfalls die Position (und Nummer) der Schlangenkörperteile als Heuristik, um den schnellsten Weg zum Ziel/anvisierten Futter zu bestimmen.

FutureSight löst das Teilproblem „Bewertung von Zügen“ und damit auch indirekt das Problem „erweiterte Kollisionsprüfung“, indem alle Bewegungsfolgen aller (der eigenen Schlange nahen)

Schlangen aus dem GameState simuliert werden, um zu ermitteln, in wie vielen Zukunftsversionen die Schlange nach einem gewählten Zug noch überlebt.

Die Funktion PanicResponse nutzt die timeout-Information aus dem GameState um im Notfall eine zufällige sichere Aktion zu wählen und löst somit das Teilproblem „einfache Kollisionsvermeidung“.

Innerhalb des Bots werden Informationen über sichere und gewählte Züge sowie Futtervorsprung und ein eingekürzter GameState ausgetauscht.

Die genutzten GameState-Informationen beschränken sich dabei auf die Informationen über das Board, also Höhe, Breite, Futterpositionen und Schlangen, wobei für letzteres die Informationen über das Aussehen der Schlange ignoriert werden. Sämtliche Informationen über das Spiel an sich werden, mit Ausnahme von timeout, nicht verwendet.

Die Information über sichere Aktionen werden an alle möglichen gewählten Strategien übertragen, sodass bei der Bestimmung des nächsten Zuges Berechnungen nicht für ohnehin tödliche Aktionen ausgeführt werden, was auch die Berechnungszeit verkürzt.

Andere ausgetauschte Informationen werden nur vom Manager genutzt, um entweder mithilfe des Futtervorsprungs eine Strategie zu wählen (Routing, falls Vorsprung zu mindestens einem Futterstück, sonst FutureSight bzw. PanicResponse, falls Zeit knapp) oder die an den Server gesendete Ausgabe („move“: *von der Strategie zurückgegebene Aktion*) zu berechnen und zu senden.

Die verschiedenen Komponenten des Systems verwenden verschiedene Arten von Agenten, um Ausgaben zu erzeugen. Die einzige erwähnenswerte Ausnahme ist die FoodLead-Funktion. Diese ist im Wesentlichen nur eine Berechnung des Vorsprungs der eigenen Schlange zu den verschiedenen Futterstücken auf dem Feld und verwendet außer der Manhattan-Distanz keine in der Vorlesung behandelten Konzepte.

Der Manager kann als modellbasierter Reflex-Agent betrachtet werden. Es werden zunächst CollisionDetection und FoodLead aufgerufen und anhand der gelieferten Ergebnisse (dem Modell) wird eine Strategie (einzige Möglichkeit, Routing, FutureSight, PanicResponse) ausgewählt, deren Rückgabe dann die endgültige Antwort des Agenten ist. Unter PanicResponse ist hierbei die zufällige Auswahl einer sicheren Aktion, falls die Antwortzeit fast um ist, zu verstehen. Der Manager stellt sicher, dass die bestmögliche Strategie gewählt wird, bzw. im Notfall zumindest eine vermeidbare Kollision vermieden wird.

Bei der CollisionDetection handelt es sich um einen Reflex Agenten, welcher durch simple Betrachtung der umliegenden Felder (Moore-Nachbarschaft mit 1 zusätzlichen Feld in jede gerade Richtung) eines Schlangenkopfes ein Feld der sicheren Aktionen berechnet. Somit ist dieser Algorithmus schnell und fehlerhafte Ausgaben sind ausgeschlossen.

Die Routenplanung ist ein planender Agent, welcher mithilfe des A*-Algorithmus den schnellsten Weg zu einem gewünschten Feld findet. Als Heuristik wird hierbei die Manhattan-Distanz zu dem Ziel verwendet, gegebenenfalls erhöht um die Zeit, bis ein existierendes Hindernis (ein Schlangenkörperteil) von einem besuchbaren Feld verschwindet. Dieser Algorithmus kann relativ schnell viele Züge im Voraus bestimmen wobei das Ergebnis die schnellste/beste Route ist. Futter kann also gezielt angesteuert werden.

Die FutureSight Funktion ist grundsätzlich ein modellbasierter Reflex-Agent. Zunächst wird mithilfe einer Evaluations-Funktion jeder sicheren Bewegung eine Wahrscheinlichkeit zugeordnet, im Verlauf der folgenden 2 bis 4 Runden zu sterben. Hierfür wird für alle möglichen eigenen und gegnerischen Züge bestimmt, ob die eigene Schlange überlegt. Mithilfe dieses Modells wird die Bewegung gespeichert, die die höchste Überlebenswahrscheinlichkeit hat. Sollte es mehrere Bewegungen mit demselben Evaluationsergebnis geben, wird die Aktion gewählt, die den Abstand zum nächsten Futterstück minimiert, bzw. eine zufällige minimierende Aktion, falls mehrere Aktionen die selbe Futterabstandsminimierung bewirken.

Die FutureSight-Funktion bewirkt eine erweiterte Kollisionsvermeidung und liefert daher grundsätzlich den sichersten Zug. Da sie jedoch sehr zeitaufwändig ist und nicht zwangsläufig vor Ablauf der Antwortzeit ein Ergebnis liefert, wird sie nur genutzt, falls es keine andere sinnvolle Strategie (nächsten Zug überleben oder Routenplanung) gibt.

Warum die genannten Ansätze entschieden wurden ergibt sich im Wesentlichen aus den erhaltenen Fähigkeiten des Bots. Eine gute Battlesnake sollte ihre Strategie an den Zustand des Spiels anpassen (Manager) und dabei zumindest eine einfache Kollisionsvermeidung beherrschen (CollisionDetection). Das Anvisieren von und Hinbewegen zu Futter ist nur sinnvoll, wenn das Futterstück auch wirklich zuerst erreicht werden kann (FoodLead), wobei Futter schnellstmöglich erreicht werden sollte (Routing), um die eigene Schlangenlänge schnell zu erhöhen und somit vor Head-to-Head Angriffen anderer Schlangen geschützt zu sein. Wenn kein Vorsprung vorliegt, ist die sinnvollste andere Strategie, den Zug zu wählen, der die Überlebenswahrscheinlichkeit innerhalb der nächsten Züge maximiert (FutureSight), um Zeit zu schinden, bis irgendwann ein Vorsprung ermittelt werden kann.

Eine Darstellung des Agentensystems lässt sich wie folgt charakterisieren:

Im Rahmen des Performance Measure soll der Bot folgende Werte maximieren:

- überlebte Runden
- 1/Kollisionen
- Gesammeltes Futter
- rechtzeitige Antworten

Für die Environment gelten folgende Eigenschaften:

- Voll observierbar, da der gesamte Spielfeldzustand zu Beginn jeder Entscheidung gesendet wird
- Stochastisch, da Futterstücke zufällig auf dem Spielfeld spawnen (zeitlich und räumlich)
- Episodisch, da vor jeder Entscheidung der aktuellste Spielzustand geliefert wird (je nachdem welche Strategien verwendet werden, könnte die Umgebung jedoch auch als sequentiell betrachtet werden)
- Dynamisch, da der nächste Zug gestartet wird (sich also das Spielfeld aktualisiert), wenn der Bot zu lange überlegt, innerhalb eines Zuges ist die Umgebung jedoch statisch
- Diskreter Beobachtungsraum, da das Feld endlich viele Positionen hat und es für jede dieser Positionen nur endlich viele Zustände (Futter, Schlange/Kopf, Hindernis, nichts) gibt
- Kompetitiv Multi-Agent, da es mehrere Schlangen gibt, die (vermutlich) alle gewinnen wollen

Die Schlange beherrscht die Aktionen:

- up, down, left, right: Bewegung ein Feld in die entsprechende Richtung
- shout: Ausgabe eines Strings, der im nächsten Zug an alle anderen Schlangen gesendet wird
 - o diese Funktion nutzt unser Bot allerdings nicht

Der Bot erhält zu Beginn jedes Zuges folgende Informationen:

- ID, Karte, Quelle, Antwortzeit und Runde des Spiels
- Regeln des Spiels, bestehend aus Name, Version, und Settings, also:
 - o FutterSpawnChance, minimale Futtermenge, Hindernisschaden pro Runde
- Höhe und Breite des Feldes
- Futter- und Hindernispositionen
- ID, Name, Leben, Körper, Latenz, Kopf, Länge, Ausruf und Aussehen aller anderen Schlangen

Bei der Umsetzung der Strategie ergaben sich folgende Schwierigkeiten:

- *Bitte Herausforderungen eintragen*

Es würden sich folgende zukünftige Verbesserungen des Systems anbieten:

- Berücksichtigung von Hindernissen bei Auswahl von Bewegungen und Routenplanung
- Hinzufügen eines Idle-Zustands (kreisen um Futter) um als lange Schlange tot durch verhungern vorzubeugen
- Hinzufügen eines Attack-Zustands (jagen kürzerer oder eingesperrter Schlangen), um die Menge an Konkurrenten schnell zu reduzieren